



Experiment – 6

Name: Simranjeet Kaur

UID: 24BAI70048

Branch: CSE – AIML (CS221)

Section/Group: 24AIT_NTPP-2A

Semester: 5th

Date of Performance: / /2026

Subject: Full Stack-II

Subject Code: 24CSP-337

EXPERIMENT – 6.1

1. **AIM:** To implement paginated and sortable REST APIs for efficient and scalable data retrieval.

2. **Objective:**

1. To understand pagination and sorting concepts
2. To implement pageable APIs using Spring Data
3. To optimize data retrieval for large datasets
4. To improve API performance and usability

3. **SOFTWARE REQUIREMENTS:**

1. Java (JDK 17+)
2. Spring Boot
3. IntelliJ IDEA
4. Postman / API testing tool
5. Maven

4. **THEORY**

Pagination and sorting are important features used in REST APIs to manage and retrieve large amounts of data efficiently. Pagination divides the records into smaller pages, allowing the client to retrieve only a fixed number of records at a time instead of loading the complete dataset. This reduces data transfer and improves the response time of the application.

Sorting is used to arrange the retrieved records according to a particular field, such as UID or name. The records can be sorted in ascending or descending order depending on the user's requirement.

Spring Boot with Spring Data JPA provides built-in support for pagination and sorting through classes such as Pageable, PageRequest, and Sort. The Pageable interface specifies the page number and number of records per page, while Sort determines the field and direction used for ordering the records.

In this experiment, a REST API is developed using Spring Boot, Spring Data JPA, and an H2 database. User records are stored in the database and retrieved through API requests using pagination and sorting parameters. The API returns information such as the current page, total number of records, total pages, and the records available on that page.

Pagination and sorting help in reducing unnecessary database operations, minimizing network data transfer, improving application performance, and making REST APIs more scalable and user-friendly.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a new Spring Boot Maven project with Java 17 and add Spring Web, Spring Data JPA, and H2 Database dependencies.
2. Create a User entity with UID and Name fields and map it to the database using JPA annotations.
3. Create a UserRepository by extending JpaRepository to perform database operations.
4. Configure the H2 database and set the required Spring Boot application properties.
5. Insert sample user records into the database using a data loader.
6. Create a REST Controller with an endpoint to retrieve users.

7. Use Pageable and PageRequest to specify the page number and number of records per page.
8. Use the Sort class to sort users by UID or Name in ascending or descending order.
9. Create POST and DELETE APIs to add and remove users.
10. Create a simple HTML interface to display users and provide controls for pagination and sorting.
11. Test the API by changing the page size, page number, sorting field, and sorting order.
12. Verify that only the required records are displayed on each page and that the Previous/Next buttons work correctly.

6. CODE

```
package com.example.experiment61;
```

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

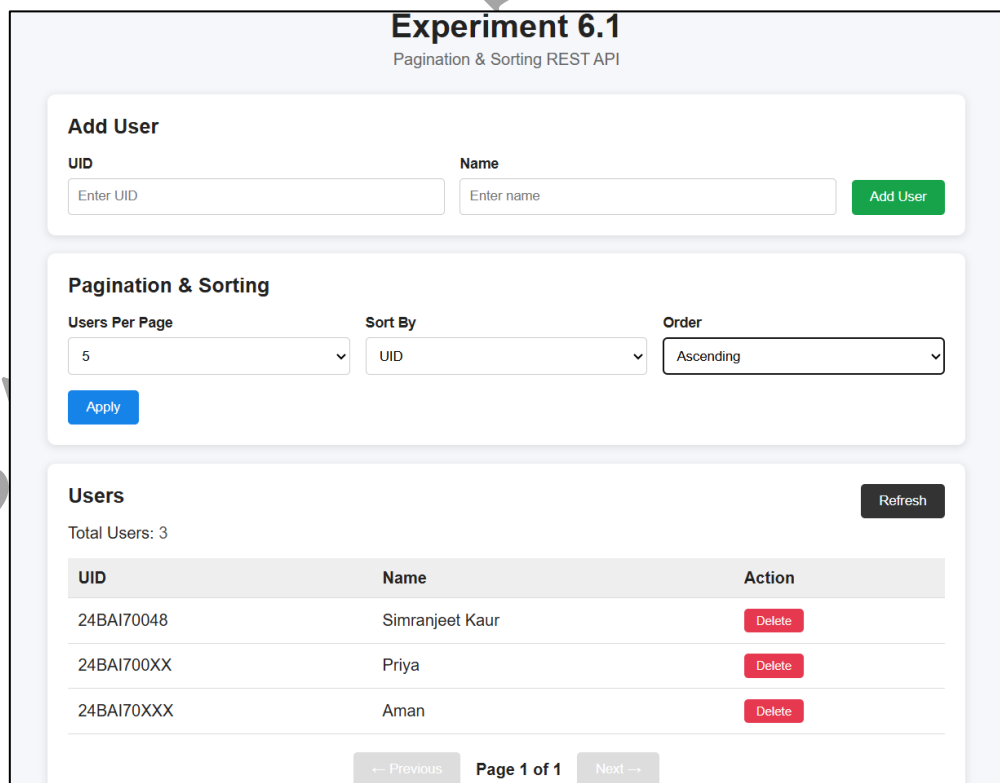
```
@SpringBootApplication
```

```
public class Experiment61Application {
```

```
    public static void main(String[] args) {
        SpringApplication.run(Experiment61Application.class, args);
    }
}
```

7. OUTPUT

The output displays a Spring Boot web interface for managing user records. It allows users to add new users, view the records in a table, and delete users. The interface also provides pagination and sorting options, through which users can select the number of records per page and sort the records by UID or Name in ascending or descending order. The Previous and Next buttons allow navigation between pages. Thus, the output demonstrates efficient retrieval and management of user data using a paginated and sortable REST API.



Experiment 6.1
Pagination & Sorting REST API

Add User

UID: Name:

Pagination & Sorting

Users Per Page: Sort By: Order:

Users

Total Users: 3

UID	Name	Action
24BAI70048	Simranjeet Kaur	Delete
24BAI700XX	Priya	Delete
24BAI70XXX	Aman	Delete

← Previous Page 1 of 1 Next →

Fig 6.1: Web Interface for Paginated and Sortable User Records

8. LEARNING OUTCOME

- Understand the concept and importance of pagination in REST APIs.
- Understand the concept of sorting and its role in data retrieval.
- Implement pagination using Spring Data JPA and Pageable.
- Implement sorting using Sort and PageRequest.
- Develop REST APIs for adding, retrieving, and deleting user records.
- Retrieve data based on different page sizes and page numbers.
- Implement sorting based on different fields such as UID and Name.
- Apply ascending and descending sorting to database records.

24BAI70048 — Simranjeet Kaur

EXPERIMENT –6.2

1. **AIM:** To optimize backend read operations using caching and efficient database query strategies..

2. **OBJECTIVE:**

1. To identify and resolve inefficient database queries
2. To understand the N+1 query problem
3. To implement caching mechanisms
4. To improve API performance through optimization

3. **SOFTWARE REQUIREMENTS:**

1. Spring Boot
2. IntelliJ IDEA
3. Database (MySQL/PostgreSQL)
4. Ehcache (or similar caching tool)
5. Apache JMeter (for performance testing)

4. **THEORY**

Backend performance is important for applications that handle a large number of database requests. Repeated database access and inefficient queries can increase response time and database load. Query optimization techniques are used to reduce unnecessary database operations and improve application performance. One common problem in JPA-based applications is the N+1 query problem. It occurs when one query is used to retrieve a set of records and additional queries are executed separately to retrieve related data for each record. This results in multiple database requests and increases execution time. The JOIN FETCH technique can solve this problem by retrieving the main entity and its related data together in a single query. This reduces database round trips and improves data retrieval efficiency.

Caching is another important optimization technique. Frequently requested data can be stored temporarily in memory so that subsequent requests can be served from the cache instead of querying the database repeatedly. Spring Boot provides caching support through annotations such as `@EnableCaching` and `@Cacheable`. For specific or complex database operations, native SQL queries can be used to directly execute SQL statements and provide greater control over data retrieval. Performance can be evaluated by measuring query execution time and comparing different approaches. In this experiment, a Spring Boot application is developed to demonstrate normal queries, JOIN FETCH, caching, and native SQL queries. The execution time of these operations is measured to understand their effect on backend performance and scalability.

5. **IMPLEMENTATION/PROCEDURE**

1. Create a new Spring Boot Maven project with Spring Web, Spring Data JPA, H2 Database, and Spring Cache dependencies.
2. Configure the H2 database and enable SQL query logging using `spring.jpa.show-sql=true`.
3. Create the User entity containing ID, UID, Name, City, and Country fields.
4. Create a Department entity and establish a relationship between User and Department using JPA.
5. Create UserRepository and DepartmentRepository by extending JpaRepository.
6. Insert sample users and departments into the database using a DataLoader.

7. Implement a normal query using `findAll()` to retrieve user records.
8. Implement a JOIN FETCH query to retrieve users and their related departments in a single database query.
9. Enable Spring caching using the `@EnableCaching` annotation.
10. Implement a cached query using the `@Cacheable` annotation to avoid repeated database access.
11. Implement a native SQL query using the `@Query` annotation with `nativeQuery=true`.
12. Create REST API endpoints for the normal, JOIN FETCH, cached, and native SQL queries.
13. Measure and display the execution time of each query.
14. Create a web interface to add, delete, sort, and display users.
15. Test each optimization technique and compare the query execution times.
16. Analyze the results to understand how JOIN FETCH and caching reduce unnecessary database operations and improve backend performance.

6. CODE

```
package com.example.experiment62;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cache.annotation.EnableCaching;

@SpringBootApplication
@EnableCaching

public class Experiment62Application {

    public static void main(String[] args) {

        SpringApplication.run(Experiment62Application.class, args);

    }

}
```

7. OUTPUT

The output displays a web-based interface for Caching and Query Optimization in Spring Boot. The interface provides an Add User form with UID, Name, City, and Country fields. It includes separate buttons for testing Normal Query, JOIN FETCH, Cached Query, and Native SQL operations. The execution time of the selected operation is displayed to evaluate its performance. Users can also be sorted by ID or Name, refreshed, and deleted from the user list. The output demonstrates how different query optimization and caching techniques can be used to improve backend data retrieval and performance.

Experiment 6.2

Caching & Query Optimization

Add User

Query Optimization Tests

Sort Users

Users loaded successfully

User List

ID	UID	Name	City	Country	Action
11	24BAI70048	Simranjeet Kaur	Rupnagar	India	Delete
12	24BAI70050	Sejal	Patiala	India	Delete
13	24BAI70521	Avneet Kaur	Mohali	India	Delete
14	24BAI70630	Minazpreet Kaur	Sangrur	India	Delete

Fig 6.2: Web Interface for Caching and Query Optimization

8. LEARNING OUTCOME

- Understand the N+1 query problem in Spring Data JPA.
- Implement JOIN FETCH to optimize database queries.
- Understand and implement caching for frequently accessed data.
- Use @Cacheable and Spring Cache to reduce repeated database access.
- Implement and execute native SQL queries for efficient data retrieval.
- Compare the execution time and performance of different query techniques.
- Identify inefficient database operations and apply suitable optimization strategies.